

PRACTICAL WEEK-3

Task-1: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Source code:

```
root@LAPTOP-1COJKKG6: ~  
GNU nano 7.2 week3pt1.c  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/types.h>  
#include <sys/wait.h>  
#include <stdlib.h>  
  
int main() {  
    pid_t pid;  
  
    printf("Parent process started\n");  
    printf("Parent PID : %d\n", getpid());  
    printf("Parent PPID : %d\n", getppid());  
  
    pid = fork();  
  
    if (pid < 0) {  
        perror("fork failed");  
        return 1;  
    }  
  
    if (pid == 0) {  
        // Child process  
        printf("Child Process\n");  
        printf("Child PID : %d\n", getpid());  
        printf("Child PPID : %d\n", getppid());  
        printf("Child State: Running\n");  
    }  
}
```

Output:

```
anuan@Anushka:~$ nano process_fork_states.c  
anuan@Anushka:~$ gcc process_fork_states.c -o process_fork_states  
anuan@Anushka:~$ ./process_fork_states  
=== Before fork() ===  
  
[Parent Process]  
PID : 516  
PPID : 336  
  
[Parent Process - Running]  
PID : 516  
PPID : 336  
Parent Process: Waiting for child to finish...  
  
[Child Process - Running]  
PID : 517  
PPID : 516  
Child Process: Sleeping for 3 seconds...  
  
[Child Process - After Sleep]  
PID : 517  
PPID : 516  
Child Process: Exiting...  
  
[Parent Process - After Child Termination]  
PID : 516  
PPID : 336  
Parent Process: Child has terminated.  
anuan@Anushka:~$
```

Observation: I learned how the fork() system call creates a child process from a parent process. I understood how to display and differentiate the PID and PPID of parent and child processes. I also learned how a parent process waits for the child process to complete using wait() and observed the different process states during execution.

Task2: The aim is to observe process state transitions such as Running, Waiting/Sleeping, and Terminated using Linux commands like ps, top, and /proc.

Source code:

```
root@LAPTOP-1COJRG6: ~  
GNU nano 7.2  
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    printf("Process started\n");  
    printf("PID : %d\n", getpid());  
    printf("PPID : %d\n", getppid());  
  
    printf("Process State: Running\n");  
  
    printf("Process State: Sleeping/Waiting\n");  
    sleep(10);  
  
    printf("Process State: Running again\n");  
  
    printf("Process State: Terminating\n");  
  
    return 0;  
}
```

Output:

```
anuan@Anushka:~$ nano process_states.c  
anuan@Anushka:~$ gcc process_states.c -o process_states  
anuan@Anushka:~$ ./process_states  
Process started.  
PID : 557  
PPID : 336  
  
Process is RUNNING...  
  
Process is SLEEPING for 20 seconds...  
Use another terminal to observe this process.
```

Observation: The process was executed and monitored using Linux processmonitoring commands. The top command was used to observe the running process, while ps was used to identify the process and its PID. The experiment showed that a process can change between **Running** and Sleeping/Waiting states during execution. The process finally terminates after completing its execution.

